



# CUDA Image Convolution

Parallel Programming for Machine  
Learning

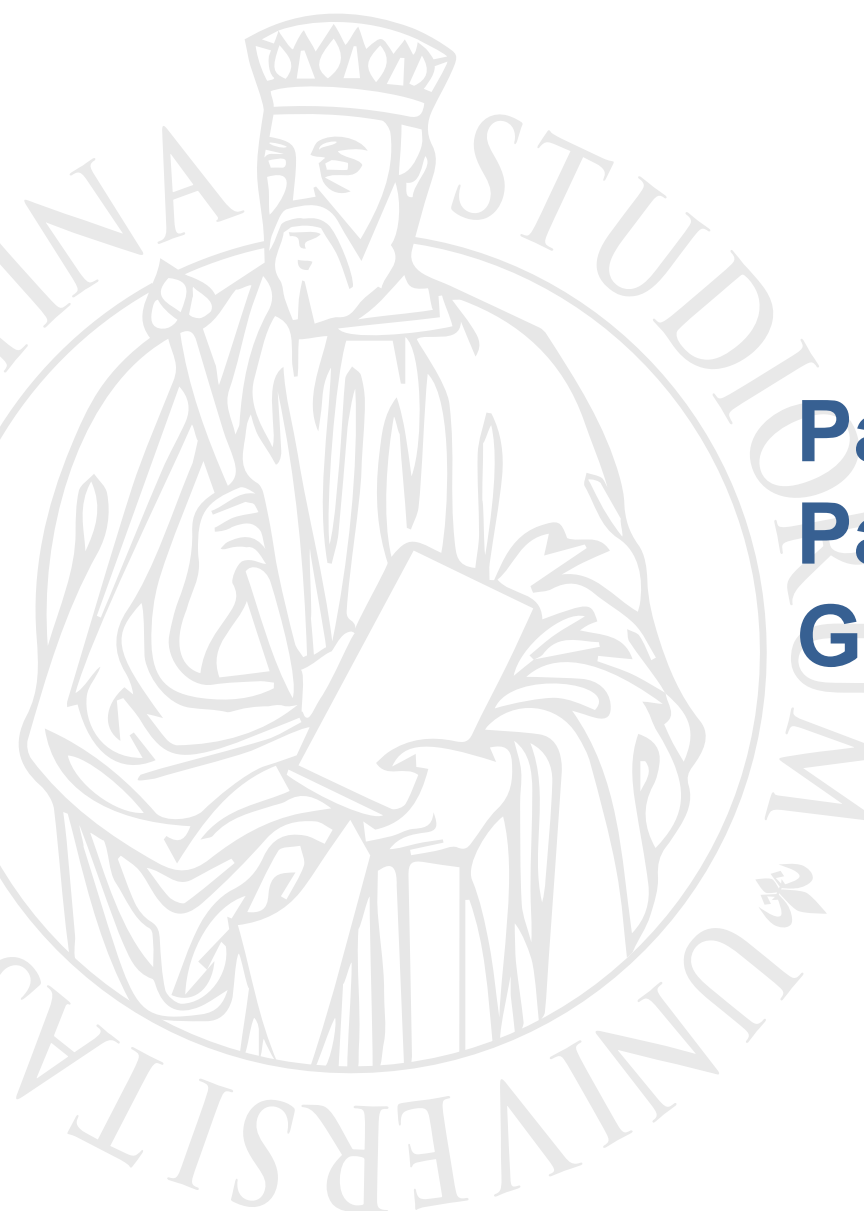
**Marco Barranco Moreno**

# Presentation Roadmap

**Part I: Core Computational Parallelism — CPU vs GPU Acceleration**

**Part II: Hardware Constraints & Advanced Memory Data Flow**





# **Part I: Core Computational Parallelism — CPU vs GPU Acceleration**

## Problem Definition: Spatial Image Convolution

- Mathematical operation sliding an  $N \times N$  matrix (3x3 Kernel) over pixels.
- **Applied Filters:** Sharpen (Edge enhancement), Box Blur (Smoothing), and Edge Detection.
- **Real-time challenge:** Massive localized matrix operations per frame.

|                                  |                                                                                  |                                                                                       |
|----------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Sharpen                          | $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$          |    |
| Box blur<br>(normalized)         | $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$  |   |
| Gaussian blur<br>(approximation) | $\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$ |  |

## Workload Scale: 4K Ultra-HD Dataset

- Forced upscaling to 4K Ultra HD resolution (3840x2160 pixels).
- **Total dataset volume:** ~8.29 Million Pixels per execution.
- **Rationale:** Standard small images fail to saturate GPU cores; 4K ensures reliable stress testing.

```
Loading internal image and upscaling to 4K (Ultra HD)...  
Resolution: 3840x2160 (8,294,400 pixels to be processed)
```



## Experimental Lab Environment

- **Device GPU:** NVIDIA Tesla T4 (Turing Architecture) | 2,560 CUDA Cores | 16 GB GDDR6.
- **Host CPU:** Intel Xeon Processor (2 Virtual Cores via Cloud Instance).
- **Software Backend:** Python 3.12, Numba 0.59, OpenCV 4.8.

| NVIDIA-SMI 580.82.07       |          |      | Driver Version: 580.82.07 |                  |                 | CUDA Version: 13.0 |             |     |
|----------------------------|----------|------|---------------------------|------------------|-----------------|--------------------|-------------|-----|
| GPU                        | Name     | Perf | Persistence-M             | Bus-Id           | Disp.A          | Volatile           | Uncorr. ECC |     |
| Fan                        | Temp     |      | Pwr:Usage/Cap             |                  | Memory-Usage    | GPU-Util           | Compute M.  |     |
|                            |          |      |                           |                  |                 |                    | MIG M.      |     |
| 0                          | Tesla T4 |      | Off                       | 00000000:00:04.0 | Off             |                    | 0           |     |
| N/A                        | 46C      | P8   | 14W / 70W                 |                  | 0MiB / 15360MiB | 0%                 | Default     | N/A |
| Processes:                 |          |      |                           |                  |                 |                    |             |     |
| GPU                        | GI       | CI   | PID                       | Type             | Process name    | GPU Memory         |             |     |
| ID                         | ID       | ID   |                           |                  |                 | Usage              |             |     |
| No running processes found |          |      |                           |                  |                 |                    |             |     |

# Compiler Stack: Just-In-Time (JIT) Compilation

- Utilizing **Numba Compiler** with the `@cuda.jit` decorator.
- Compiles high-level Python code dynamically into native PTX assembly instructions.
- Avoids interpreter latency, enabling close-to-metal hardware execution.

```
#Cell 3: The Parallel version (GPU with CUDA)
```

```
@cuda.jit
```

## CPU Baseline: The Sequential Bottleneck

- Implementation: **4 nested for loops** in Python.
- Execution path: Traversing width, height, and the 3x3 kernel matrix sequentially.
- Complexity Profile:  $O(N \times M)$  executed entirely on a single Host CPU thread.

```
@cuda.jit
def convolucion_gpu(imagen, kernel, resultado):
    x, y = cuda.grid(2)

    alto, ancho = imagen.shape
    k_alto, k_ancho = kernel.shape
    pad_y, pad_x = k_alto // 2, k_ancho // 2

    if y >= pad_y and y < (alto - pad_y) and x >= pad_x and x < (ancho - pad_x):
        suma = 0.0

        for ky in range(k_alto):
            for kx in range(k_ancho):
                pixel_img = imagen[y + ky - pad_y, x + kx - pad_x]
                pixel_kernel = kernel[ky, kx]
                suma += pixel_img * pixel_kernel

        resultado[y, x] = suma

import numba
```



## Baseline Performance & Core Issues

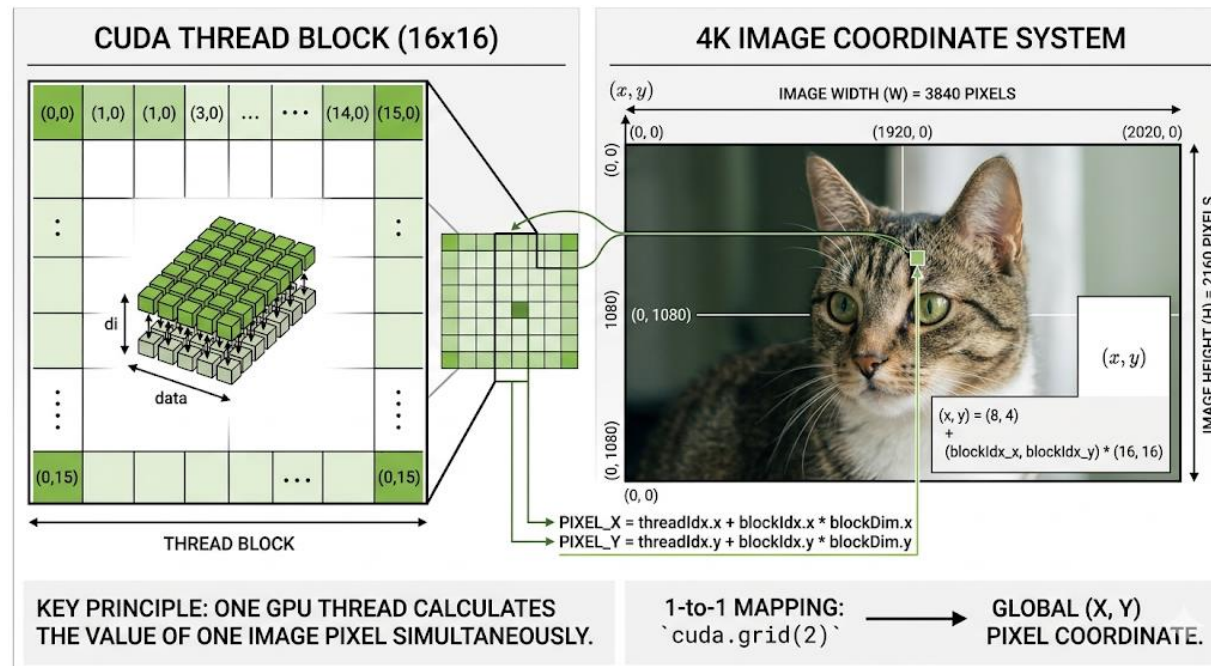
- Baseline CPU Execution Time: **41.5150 seconds**.
- Main Bottleneck: Inability of traditional single-threaded architectures to scale for high-resolution video streams.

```
Starting CPU processing (Sequential)...  
Total CPU time: 41.5150 seconds
```



## GPU Strategy: 2D Grid Mapping

- Classification: *Embarrassingly Parallel* problem (pixels calculated independently).
- Mapping paradigm: **1 GPU Thread = 1 Image Pixel**.
- Dynamic coordinate resolution using the native multi-dimensional API `cuda.grid(2)`.



## Thread Blocks & Warp Optimization

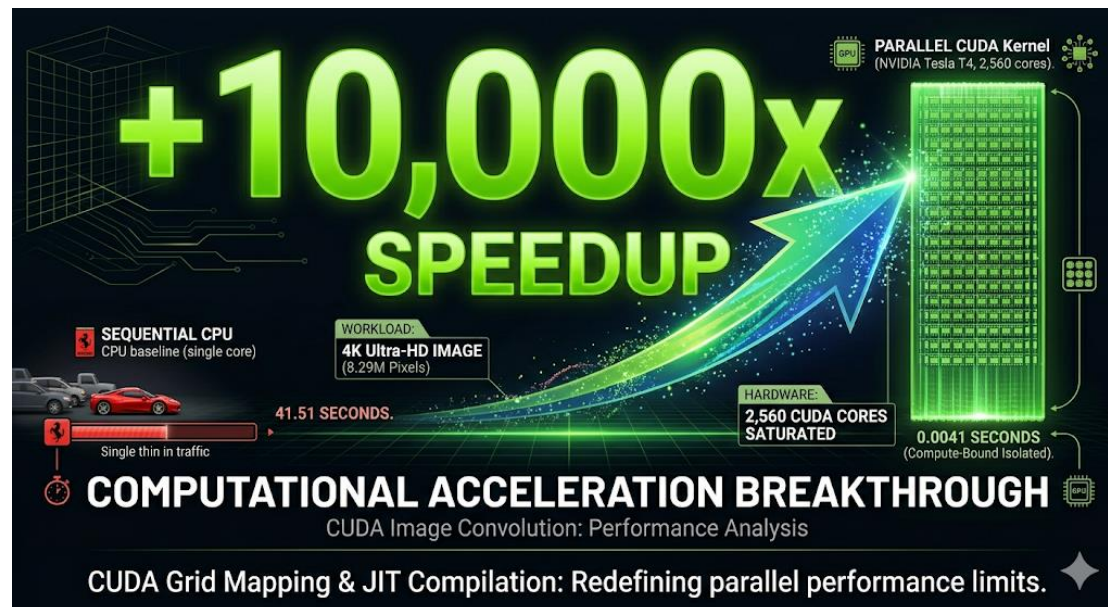
- **Execution layout:** Thread Blocks configured to **16x16 (256 threads/block)**.
- **Alignment:** Designed as an exact multiple of NVIDIA's **Warp Size (32 threads)**.
- **Benefit:** Minimizes thread divergence and ensures maximum hardware scheduler occupancy.

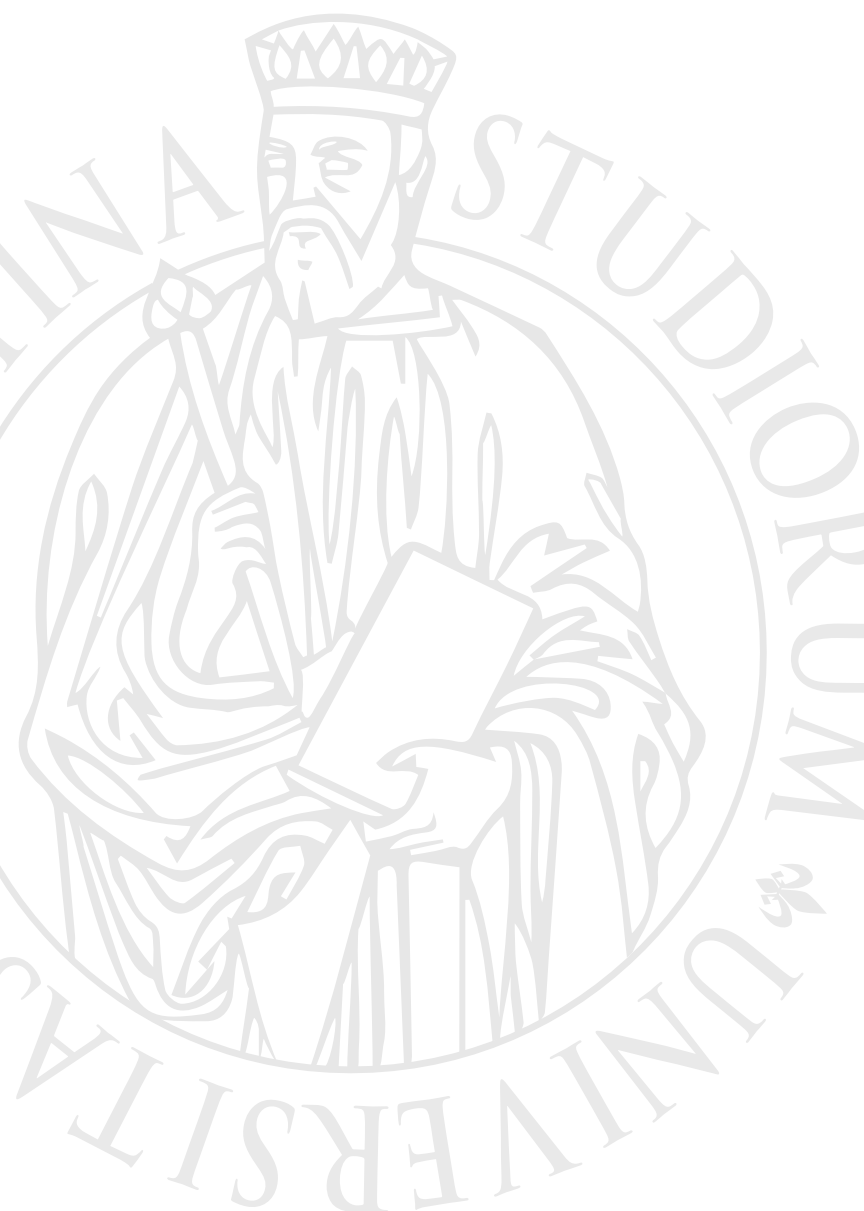
```
hilos_por_bloque = (16, 16)
bloques_x = math.ceil(ancho_4k / hilos_por_bloque[0])
bloques_y = math.ceil(alto_4k / hilos_por_bloque[1])
bloques_por_grid = (bloques_x, bloques_y)
```



## Results: Pure Computational Acceleration

- Rigorous methodology: Implementing a **JIT Warm-up run** to isolate compilation overhead.
- Pure GPU Kernel Time: **0.0041 seconds**.
- Pure Mathematical Speedup: **10,125.6x** over the CPU.





## **Part II: Hardware Constraints & Advanced Memory Data Flow**

# Memory Management: Host to Device Data Routing

- Physical reality: Image arrays do not natively reside inside the GPU core cluster.
- Low-level explicit API utilization in Numba:
  1. `cuda.to_device()`: Transferring pixel buffers to global VRAM.
  2. `cuda.device_array()`: Allocation of the output result matrix in VRAM.
  3. `.copy_to_host()`: Retrieving the modified data back to the Host RAM.



# Memory Management: Host to Device Data Routing

## #Cell 4: Execution, Benchmarking and Visualization

```
print("Starting CPU processing (Sequential)...")
inicio_cpu = time.time()
resultado_cpu = convolucion_cpu(img_gray, filtro_elegido)
tiempo_cpu = time.time() - inicio_cpu
print(f"CPU time: {tiempo_cpu:.4f} seconds")

hilos_por_bloque = (16, 16)
bloques_x = math.ceil(ancho / hilos_por_bloque[0])
bloques_y = math.ceil(alto / hilos_por_bloque[1])
bloques_por_grid = (bloques_x, bloques_y)

img_device = cuda.to_device(img_gray)
filtro_device = cuda.to_device(filtro_elegido)
resultado_device = cuda.device_array(img_gray.shape, dtype=np.float32)

convolucion_gpu[bloques_por_grid, hilos_por_bloque](img_device, filtro_device, resultado_device)
cuda.synchronize()

inicio_gpu_puro = time.time()
convolucion_gpu[bloques_por_grid, hilos_por_bloque](img_device, filtro_device, resultado_device)
cuda.synchronize()
tiempo_gpu_puro = time.time() - inicio_gpu_puro

resultado_gpu = resultado_device.copy_to_host()
resultado_gpu = np.clip(resultado_gpu, 0, 255).astype(np.uint8)
```

## Results: Full Pipeline End-to-End Execution

- End-to-End (E2E) Execution Time: **0.0850 seconds**.
- **Metric includes:** Memory Allocation + Data Upload + Kernel Compute + Data Download.
- **Real-World Pipeline Speedup: 488.4x** compared to CPU.

| Implementation Strategy   | Block Size | Resolution | Execution Time (s) |
|---------------------------|------------|------------|--------------------|
| Sequential (CPU Baseline) | N/A        | 4K         | 41.5150            |
| CUDA (End-to-End)*        | 16x16      | 4K         | 0.0850             |
| CUDA (Compute-Bound)**    | 16x16      | 4K         | 0.0041             |

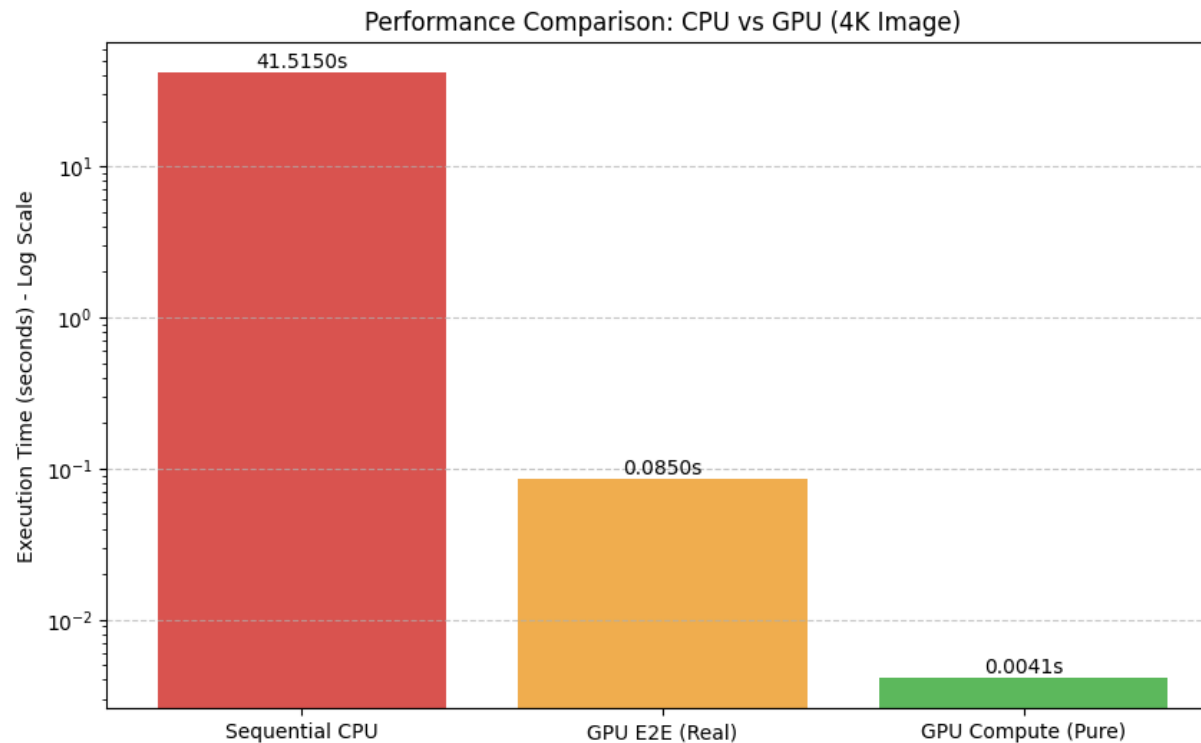


# Benchmark Analysis: Compute-Bound vs Memory-Bound

- **The Shift:** Accelerating calculations shifts the system profile from *Compute-Bound* (CPU) to *Memory-Bound* (GPU).
- **The PCIe Bottleneck:** Physical data transfer across the PCI-Express bus consumes **~95% of the total execution budget**.
- **Outcome:** Core **data starvation**. Cores calculate the matrix faster than the motherboard can feed them the pixels.



# Benchmark Analysis: Compute-Bound vs Memory-Bound



## Conclusions & Future Work directives

- **Success:** Achieved a massive 10,125x Speedup in pure computation.
- **Efficiency:** The 4K image is processed in real-time frame rates (under 100ms E2E).
- **Bottleneck:** Identified that the physical data transfer (PCIe) is the main limiting factor.
- **Visual Proof:** The output image matches the sequential baseline with 100% pixel accuracy.

1. Original 4K (Upscaled)



2. GPU Filter (Complete)



3. Filter Test (Zoom 1:1)

